

ROB301 Final Project Report

December 3rd, 2025

Melysa Choi

1009953294

Zarah Mathew

1009961720

1 Introduction

The objective of this project is to design a control system by integrating PID line-based following, dead-reckoning, and Bayesian localization techniques to simulate a robot mail delivery system. Using ROS, the project designs and tests a Turtlebot to autonomously navigate through a closed delivery loop and deliver mail to any office on the route. The map consists of eleven office locations, each marked by one of the four possible colors (red, yellow, green, and blue) with a numerical label from 2 through 12. It is connected through continuous white tape that serves as a path for line following. The topological map is shown below:

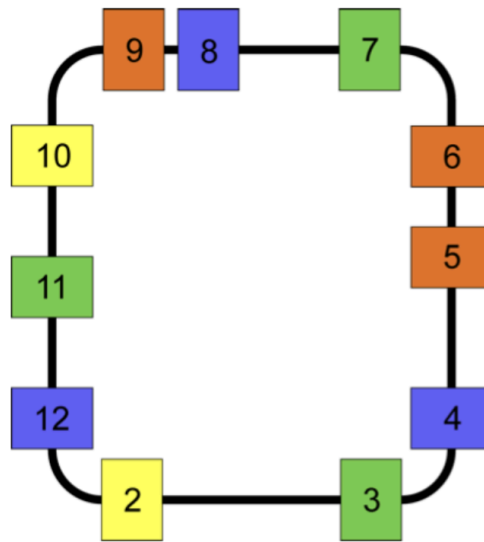


Figure 1: Topological map of the mail delivery route [1]

As multiple offices share the same colour, deterministic localization is not sufficient for the robot to recognise its location; instead, it must build a belief distribution over possible locations and refine this belief continuously using camera sensing. Thus, Bayesian localization is central for this project. The Turtlebot must integrate accurate line following, color detection, and state estimation to execute full proof-of-concept demonstration where the robot begins at an arbitrary location and delivers mail to three randomly selected offices with limited intervention.

2 Brief Description of the Robot Platform

The Turtlebot 3 Waffle Pi, a lightweight differential-drive robot that is capable of precise linear and angular control, is utilized for this project. Low-level commands are issued via ROS through the `/cmd_vel` topic [2]. Its architecture consists of the following:

OpenCR 1.0 and Dynamixel XM430: Uses 2 dynamixel actuators to drive wheels which are controlled by the OpenCR 1.0 microcontroller [2].

Lidar Sensor: Equipped with 2D 360° lidar (light detection and ranging) sensor. It is capable of sensing obstacles with a scan rate of 300 ± 10 rpm and angular resolution of 1° . The output is an array length of 360. This is useful in path tracking and detecting color for office identification in our lab [2].

Camera: Uses a color sensor to distinguish the office location and the path marked with tape on the map. It outputs two values: the index with the greatest light intensity in the camera image array (published via the /line_idx) to sense the line and an RGB value (published via the /mean_img_rgb) to detect mean pixel intensity in the current image [1].

Raspberry Pi 3B: Used as the Turtlebot PC and is responsible for executing ROS nodes, processes camera input, and publishes sensory data to ROS topics. It is also responsible for receiving commands from the remote PC and commanding corresponding actuators or sensors [2].

3 Overview of Solution Strategy

3.1 Bayesian Localization

Bayesian Localization applies the Bayes filter algorithm which allows the robot to estimate its position by keeping a belief distribution:

$$bel(x_k) = p(x_k | z_{1:k}, u_{0:k-1})$$

It gives a probability of each position where the robot may be, given all actions and measurements taken so far. As more steps are taken, the more confident the prediction model becomes.

For this project, the Bayesian localization is used to determine which office the robot is most likely in, given the current color the sensor measured and previous measurements. In this section, this logic is simulated by representing the topological map as an array, applying the motion model (the transition matrix) at each step, then given a measured color at each step, generating a prediction using the Bayes filter. Using the given data in the attachment of the final project handout [1], the prediction distribution at each step is shown in Figure 2.

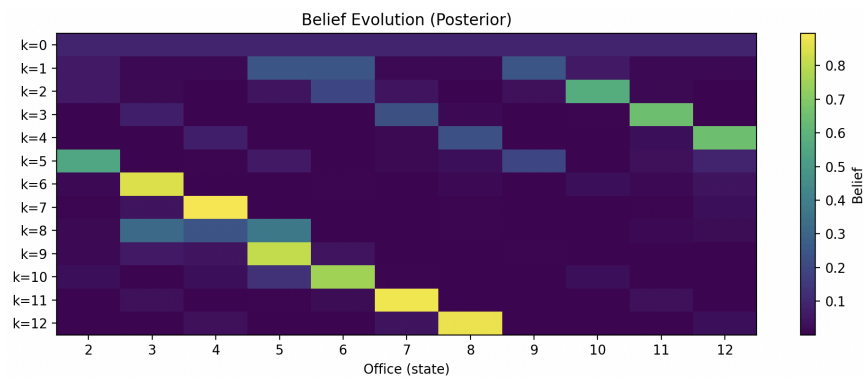


Figure 2: Prediction belief distribution of each office at each step k

3.2 Full Route Execution

For this section, the robot navigates through the circular route and stops when it returns to its starting office. To achieve this, the robot has to complete line following through PID controls, drive straight when it reaches an office, be able to detect whether it is in an office by matching the color sensor data to the color code given, keep an “in-office” status, and count how many offices it has passed through.

The line following was achieved using the algorithm for Lab 2, with calibrated k_p , k_i , and k_d values. To determine whether the robot was in an office, it continuously compared the present color sensor data to the previous data. If there exists an obvious color difference, the algorithm assumes that it has entered or left an office. Additionally, it also matched the color sensor to the color array given, ensuring it was able to detect whether it was on a line or in an office. For instance, if the “in-office” status is False and it detects a color within the color array that is not the line, it registers that it has entered an office and adds it to the office counter. Once it has traversed 12 offices, the robot stops (by setting the linear and angular velocity to zero).

3.3 Bayesian Localization Implementation

Building on the previous section, Bayesian localization was implemented to predict which exact office it may be in. At each office, it took in the color sensor data and compared it to the motion model and the topological map, then generated a probability of what office it is most likely to be in, given previous measurements and predictions. Figure 3 shows the prediction model at each office it entered, with the robot starting at office 2.

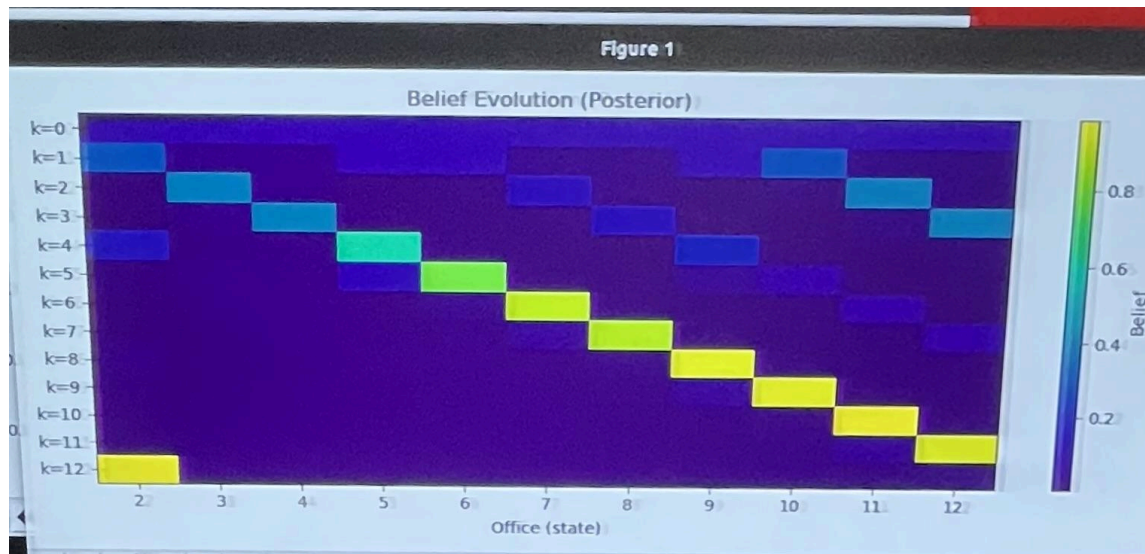


Figure 3: Prediction belief distribution of each office k approached

3.4 Demonstration of Navigation

To truly apply this algorithm for delivery navigation, various offices were assigned as waypoints to stop at. As seen from Figure 3, the prediction model is less confident during the early steps. Therefore, in order not to stop at the wrong office, the algorithm will only stop if

there exists a unique most likely office (based on the probability distribution) that has a confidence rate greater than 0.50.

4 Technical details on the design methodology

4.1 Colour Recognition

Given the limitations of the actual map, room lighting, and other external factors, the color code given has a relatively drastic deviation from the actual colors. To tackle this, each color was recalibrated using the color sensor data collected while hovering the robot over each color. A threshold was also created to account for fluctuating color values, such that if the color detected falls within the range, it is registered as an office color. However, due to some colors having relatively similar R, G, or B, values, each color was given a different threshold. For instance, the robot would often mistake green or the line for yellow. To account for this, the tolerance for yellow is significantly smaller than green or the line, with the line having the largest tolerance, such that any color that's not one of the office colors would be considered following the line. The calibration and tolerance are shown in Table 1.

	Red	Green	Blue	Yellow	Line
Color Code	[250.12, 145.42, 89.75]	[169.85, 179.76, 171.13]	[199.27, 133.04, 178.41]	[176.94, 157.06, 137.69]	[150, 150, 150]
Tolerance	± 32	± 26	± 34	± 12	± 999

Table 1: Calibrated color code and tolerance threshold for each office color and line

4.2 PID control when traversing an office

It was observed that when the robot first approaches an office, it becomes jittery and tends to go straight in a direction off track. This is a result of frame delay, where by the time the robot establishes that it is in an office, the path has been subjected to enough inaccurate PID calibration that it completely deviates off the route. To account for this, the rospy rate was increased to 90, which saw significant changes. Fine-tuning was also done by keeping a history of the last 10 movements the robot has done, where by the time it registers it is in an office, it inverses that set of movements to put it back on track before going in a straight line.

5 Summary of Demonstration performance

In the final demonstration of the project, the robot successfully performed the mail delivery task at three random office locations when placed at an arbitrary location and demonstrated correct convergence of the Bayesian filter after passing a few offices. Only one manual correction was needed as the Turtlebot was navigating through the route. Line following maintained stability when following the white tape, even along curved sections of the route, and was executed smoothly. However, it had issues following the line after reaching an office. This may be because of the delay in the robot's control loop responding to a change in colour, causing

the robot to momentarily deviate when exiting an office. Overall, the system met the demonstration requirements and reliably followed the mail delivery route.

6 List of potential improvements in the future

Although the robot successfully followed the mail delivery route, its performance can be improved.

6.1 Improved Line Following

As stated in Section 5, the line following algorithm had difficulty recognizing the white tape when exiting offices, leading the robot to drift before re-acquiring its path. This can be improved by reducing the linear velocity and increasing the controller gain when a transition from “office color” back to “white tape” is detected. This would give the Turtlebot more time to correct its heading and follow the line path more accurately.

6.2 Improved Motion and Measurement Model Estimation

The state model and measurement model in the algorithm uses predefined probabilities as stated in the handout. This caused inconsistencies in localization performance, especially since lighting conditions and motion variations in the lab differed from the idealized simulation model. Thus, collecting real transition data from the robot as it traverses through the route over multiple trials and normalizing these probabilities could provide more accurate results.

7 Conclusion

In conclusion, our team was able to demonstrate our knowledge of PID line following and Bayesian localization by successfully implementing an autonomous mail delivery robot that accurately followed the route, identified office locations, and executed the required delivery stops during the final demonstration with minimal intervention. Through iterative tuning of colour thresholds, PID gains, and control logic, the algorithm achieved stable line tracking and accurate Bayesian convergence. While performance was strong as a whole, further improvements such as refining line following and motion, and measurement models could increase the reliability of the robot. Ultimately, the robot completed the route successfully and met all project objectives.

8 References

- [1] University of Toronto , “Final Design Project: The Galbraith Memorial Mail Robot,” 2025.
- [2] University of Toronto , “Laboratory 0: Meet your TurtleBot 3 Waffle Pi,” 2025.

9 Appendix

9.1 Code for final project

```
#!/usr/bin/env python

import rospy

import math

from geometry_msgs.msg import Twist

from std_msgs.msg import UInt32, Float64MultiArray

import numpy as np

import colorsys

import matplotlib.pyplot as plt

class BayesLoc:

    def __init__(self, p0, colour_codes, colour_map):

        self.colour_sub = rospy.Subscriber(

            "mean_img_rgb", Float64MultiArray, self.colour_callback

        )

        self.line_sub = rospy.Subscriber("line_idx", UInt32, self.line_callback)

        self.cmd_pub = rospy.Publisher("cmd_vel", Twist, queue_size=1)

        self.num_states = len(p0)          # 11 offices (2..12)
        self.states = np.arange(2, 12)     # office numbers from 2 to 12
        assert self.num_states == len(self.states)

        self.colour_codes = np.array(colour_codes, dtype=float)
        self.colour_map = colour_map
        self.probability = np.array(p0, dtype=float)
        self.state_prediction = np.zeros(self.num_states)

        self.topology_colors = [
            "yellow", "green", "blue", "orange",
            "orange", "green", "blue", "orange",
            "yellow", "green", "blue",
        ]
        assert len(self.topology_colors) == self.num_states

        self.motion_table = {
            -1: {-1: 0.85, 0: 0.10, +1: 0.05},
            0: {-1: 0.05, 0: 0.90, +1: 0.05},
            1: {-1: 0.05, 0: 0.10, +1: 0.85},
        }
        # Transition Matrices
        self.T_u = {u: self.state_model(u) for u in (-1, 0, 1)}
```

```

# Measurement Model
self.color_names_sim = ["blue", "green", "yellow", "orange"]
self.color_to_idx_sim = {c: i for i, c in enumerate(self.color_names_sim)}
self.Pz_given_color = {
    "blue":    [0.60, 0.20, 0.05, 0.05],
    "green":   [0.20, 0.60, 0.05, 0.05],
    "yellow":  [0.05, 0.05, 0.65, 0.20],
    "orange":  [0.05, 0.05, 0.15, 0.60],
    "nothing": [0.10, 0.10, 0.10, 0.10],
}

self.belief_history = [self.probability.copy()]

self.cur_colour = None # [r, g, b]
self.line_index = None

def colour_callback(self, msg):

    self.cur_colour = np.array(msg.data) # [r, g, b]

def line_callback(self, msg):

    self.line_index = msg.data

def wait_for_colour(self):

    rate = rospy.Rate(100)

    while not rospy.is_shutdown() and self.cur_colour is None:

        rate.sleep()

def state_model(self, u):

    T = np.zeros((self.num_states, self.num_states), dtype=float)
    probs = self.motion_table[u]
    for j in range(self.num_states):
        for delta, p in probs.items():
            i = (j + delta) % self.num_states
            T[i, j] += p
    return T

def _classify_measurement(self):

    if self.cur_colour is None:
        self.wait_for_colour()

    dists = [np.linalg.norm(self.cur_colour - code) for code in self.colour_codes]
    idx = int(np.argmin(dists))

    if idx == 4:
        return "nothing"

    if idx == 0:

```



```

        return "orange"
    if idx == 1:
        return "green"
    if idx == 2:
        return "blue"
    if idx == 3:
        return "yellow"

    return None

def measurement_model(self):

    z = self._classify_measurement()

    if z is None:
        return np.ones(self.num_states, dtype=float)

    L = np.zeros(self.num_states, dtype=float)

    # Crosscheck with calibrated colors
    for i, true_color in enumerate(self.topology_colors):
        col_idx = self.color_to_idx_sim[true_color]
        L[i] = self.Pz_given_color[z][col_idx]

    return L

def state_predict(self, u=1):
    self.state_prediction = self.T_u[u] @ self.probability

def state_update(self):

    L = self.measurement_model()
    bel_post = self.state_prediction * L

    s = bel_post.sum()
    if s > 0:
        bel_post /= s
    else:
        bel_post = np.ones(self.num_states, dtype=float) / self.num_states

    self.probability = bel_post
    self.belief_history.append(self.probability.copy())

    # Print Full probability array: (office_number, probability)
    prob_array = [(int(st), float(p)) for st, p in zip(self.states, self.probability)]
    rospy.loginfo("[Bayes] Belief over offices (office, prob):")
    rospy.loginfo(str(prob_array))

    # Find all offices with the maximum probability at every office encountered
    max_prob = np.max(self.probability)
    tol = 1e-6
    best_indices = np.where(np.isclose(self.probability, max_prob, atol=tol))[0]
    best_offices = [int(self.states[i]) for i in best_indices]

    if len(best_offices) == 1:
        rospy.loginfo(
            f"[Bayes] Most likely office: {best_offices[0]} "
            f"(prob = {max_prob:.3f})"

```

```

    )
else:
    rospy.loginfo(
        f"[Bayes] Most likely offices (tie): {best_offices} "
        f"(prob = {max_prob:.3f})"
    )

def plot_beliefs(self):
    if len(self.belief_history) == 0:
        return

    bhist = np.array(self.belief_history)

    plt.figure(figsize=(10, 4))
    plt.imshow(bhist, aspect='auto', interpolation='nearest')
    plt.colorbar(label='Belief')
    plt.yticks(range(len(bhist)), [f"k={i}" for i in range(len(bhist))])
    plt.xticks(range(self.num_states), self.states)
    plt.xlabel("Office (state)")
    plt.title("Belief Evolution (Posterior)")
    plt.tight_layout()

    # Final belief bar plot
    plt.figure(figsize=(8, 4))
    plt.bar(self.states, bhist[-1])
    plt.xlabel("Office (state)")
    plt.ylabel("Belief")
    plt.title("Final Belief (after last office)")
    plt.tight_layout()

    plt.show()

def deliverable_2(self, stop_office):

    colour_codes = [
        [250.12, 145.42, 89.75], # red
        [169.85, 179.76, 171.13], # green
        [199.27, 133.04, 178.41], # blue
        [176.94, 157.06, 137.69], # yellow
        [150, 150, 150], # line
    ]

    colour_names = ["red", "green", "blue", "yellow", "line"]
    COLOUR_TOLERANCE = {
        0: 32, # red
        1: 26, # green
        2: 34, # blue
        3: 12, # yellow
        4: 999, # line
    }

    ENTER_OFFICE_FRAMES = 5
    EXIT_ON_LINE_FRAMES = 3
    EXIT_ON_NONOFFICE_FRAMES = 8

    rate = rospy.Rate(90)

```

```

center = 640 // 2
kp = 0.003125
ki = 0.000001
kd = 0.000001

integral = 0
lasterror = 0

office_count = 0
in_office = False

office_detect_count = 0
line_detect_count = 0

non_office_count = 0

# Undo redundant PID corrections
last_angular_cmds = []
max_history = 10

try:
    while not rospy.is_shutdown():

        if self.cur_colour is None or self.line_index is None:
            rate.sleep()
            continue

        distances = [
            np.linalg.norm(self.cur_colour - np.array(c))
            for c in colour_codes
        ]

        ''' Check whether robot is in an office given color data'''
        detected_colour = np.argmin(distances)
        min_dist = distances[detected_colour]

        twist = Twist()
        is_office_colour = detected_colour < 4
        detected_is_line = (detected_colour == 4)
        threshold = COLOUR_TOLERANCE[detected_colour]

        detected_is_valid_office = (
            is_office_colour and (min_dist < threshold)
        )

        detected_any_office = is_office_colour

        if detected_is_valid_office:
            office_detect_count += 1
        else:
            office_detect_count = 0

        if detected_is_line:
            line_detect_count += 1
        else:
            line_detect_count = 0

        if (not detected_any_office) and (not detected_is_line):
            non_office_count += 1

```

```

else:
    non_office_count = 0

''' ENTERING AN OFFICE '''

if (office_detect_count >= ENTER_OFFICE_FRAMES) and (not in_office):
    in_office = True
    office_count += 1

    integral = 0
    lasterror = 0

    rewind_steps = len(last_angular_cmds)

    rospy.loginfo(f"Entered office count:{office_count}
({colour_names[detected_colour]}")

    self.state_predict(u=1)
    self.state_update()

    max_prob = np.max(self.probability)
    tol = 1e-6
    best_indices = np.where(np.isclose(self.probability, max_prob, atol=tol))[0]

    # Stop at this office if there exists a unique best office with a confidence > 0.5
    if len(best_indices) == 1 and max_prob > 0.5:
        est_office = int(self.states[best_indices[0]])
        if est_office in stop_office:
            rospy.loginfo(
                f"REACHED DESIRED OFFICE: {est_office}. "
                f"Stopping for 3 seconds."
            )

            forward_twist = Twist()
            forward_twist.linear.x = 0.07
            forward_twist.angular.z = 0.0

            fwd_rate = rospy.Rate(20)
            t0 = rospy.Time.now()
            while (
                not rospy.is_shutdown()
                and (rospy.Time.now() - t0).to_sec() < 0.5
            ):
                self.cmd_pub.publish(forward_twist)
                fwd_rate.sleep()

            stop_twist = Twist()
            stop_twist.linear.x = 0.0
            stop_twist.angular.z = 0.0

            stop_rate = rospy.Rate(10)
            t0 = rospy.Time.now()

            while (
                not rospy.is_shutdown()
                and (rospy.Time.now() - t0).to_sec() < 3.0
            ):
                self.cmd_pub.publish(stop_twist)
                stop_rate.sleep()

```

```

        office_detect_count = 0
        line_detect_count = 0
        non_office_count = 0

''' EXITING THE OFFICE'''

if in_office:
    # if detect a line
    exit_due_to_line = (line_detect_count >= EXIT_ON_LINE_FRAMES)

    # if detect a color thats not the previous office color;
    # this covers for when PID drifts off track or when it detects outside of the office

and the line

    exit_due_to_nonoffice = (non_office_count >= EXIT_ON_NONOFFICE_FRAMES)

    if exit_due_to_line or exit_due_to_nonoffice:
        in_office = False
        rospy.loginfo(
            f"Exited office count:{office_count} "
            f"(exit_due_to_line={exit_due_to_line}, "
            f"exit_due_to_nonoffice={exit_due_to_nonoffice})"
        )

        office_detect_count = 0
        line_detect_count = 0
        non_office_count = 0

    if in_office:
        # Keep driving straight
        twist.linear.x = 0.07
        twist.angular.z = 0.0

        # Reset PID controls to minimize unnecessary deviations
        last_angular_cmds.clear()
        rewind_steps = 0

        self.cmd_pub.publish(twist)

else:
    ''' LINE FOLLOWING '''
    error = self.line_index - center
    derivative = error - lasterror

    integral = integral + error

    lasterror = error

    twist = Twist()
    twist.linear.x = 0.06
    pid_cmd = (-kp* error) + (-ki*integral) +(kd*derivative)
    twist.angular.z = pid_cmd
    self.cmd_pub.publish(twist)

    # keep movement history for rewind
    last_angular_cmds.append(pid_cmd)
    if len(last_angular_cmds) > max_history:
        last_angular_cmds.pop(0)

```

```

        rate.sleep()

    except KeyboardInterrupt:
        ''' Safety Stop (incase it doesn't stop after Ctrl + C)'''
        stop = Twist()
        stop.linear.x = 0.0
        stop.angular.z = 0.0
        self.cmd_pub.publish(stop)
        rospy.sleep(0.1)
        rospy.loginfo("Robot stopped. ")

    self.plot_beliefs()

if __name__ == "__main__":

    colour_map = [3, 0, 1, 2, 2, 0, 1, 2, 3, 0, 1]

    colour_codes = [
        [250.12, 145.42, 89.75], # red
        [169.85, 179.76, 171.13], # green
        [199.27, 133.04, 178.41], # blue
        [176.94, 157.06, 137.69], # yellow
        [150, 150, 150], # line
    ]

    p0 = np.ones_like(colour_map) / len(colour_map)

    rospy.init_node("final_project")
    rospy.sleep(0.5)
    stop_office = [5, 7, 9]

    controller = BayesLoc(p0, colour_codes, colour_map)
    controller.deliverable_2(stop_office)

```